2013 Seventh International Conference on IT Security Incident Management and IT Forensics

# Forensic Application-Fingerprinting based on File System Metadata

Sven Kälber Andreas Dewald Felix C. Freiling
*Department of Computer Science*
*Friedrich-Alexander-University*
*Erlangen, Germany*
{*sven.kaelber, andreas.dewald, felix.freiling*}*@cs.fau.de*

**Abstract**

While much work has been invested in tools for aquisition and extraction of digital evidence, there are only few tools that allow for automatic event reconstruction. In this paper, we present a generic approach for forensic event reconstruction based on digital evidence from file systems. Our approach applies the idea of fingerprinting to changes made by applications in file system metadata. We present a system with which it is possible to automatically compute file system fingerprints of individual actions. Using NTFS timestamps as an example, we show that with our approach it is possible to automatically reconstruct actions performed by different applications even if the set of files accessed by those actions overlap.

**Keywords**

Application Fingerprinting; Digital Investigation; Digital Forensics; Event Reconstruction

## I. Introduction

### A. Motivation

In addition to the increasing number of digital investigations, today's investigators are also confronted with a remarkable increase of the data volume that needs to be processed. Analyzing this amount of digital evidence is one of the major challenges that digital forensics is currently faced with [1], [2]. Apart from being more selective in the aquisition process [3], [4], abstraction and automation are considered necessary to survive the data deluge [1]. Still, many aspects of a digital investigation remain a manual and cumbersome task because most tools mainly focus on supporting an investigator in the aquisition and extraction phases of digital evidence in common process models [5], [6]. Methods supporting the *event reconstruction* phase [7] have not progressed with the same pace and seldomly extend the possibilities of timelining.

### B. Related Work

Generally, automatic event reconstruction is a hard task because there can be so many different types of events. For specific situations and contexts, automated event reconstruction can however be performed. For example, Zhu et al. [8] present the idea of reconstructing user activities based on *ShellBag* information stored in the Windows Registry. Commonly, this information is used by the operating system to store user preferences, such as window sizes and positions. Zhu et al. performed different actions, like opening and closing directories in the Windows Explorer, in a virtual machine running Windows XP. By analyzing the changes to the ShellBag information afterwards, they were able to identify nine rules about the correlation between user activities performed in the Windows Explorer and changes to the ShellBags. By testing those rules on an analyzed system's ShellBag information, the investigator is able to reconstruct the latest Windows Explorer activities.

As another example, Carvey [9] proposed an approach for reconstructing user activities based on information stored in a user's `NTUSER.DAT` file. This file file holds all user specific Windows Registry settings. Carvey shows, that for a set of predefined actions, it is possible to reconstruct events such as typing a command in the *Start* → *Run* box of the Windows Start Menu, accessing specific objects on the system, or connecting to a wireless network, for example.

Chow et al. [10] analyzed the modifications to timestamp values that are the result of commonly used operations within the Windows Explorer, such as copying, modifying, deleting and accessing files and directories stored in the file system. Based on those modifications they defined seven rules to support an investigator to tell for example if a file was copied to the system or if a file was opened/viewed by the user. Grier [11], however, focused on reconstructing the copying process of files and folders. Unlike Chow et al., he aims at detecting if a folder containing multiple files was copied from the investigated system to another location, even if the destination is unknown. He found that the act of copying files creates something he calls a *cutoff cluster*. A cutoff cluster is a conspicuous amount of files (cluster) stored in the investigated folder that have their access timestamp set to the time when the folder was actually copied and no files within the folder have their access timestamp set to a value previous to the copying event (cutoff). He has shown that it is possible to tell if and when a folder was last copied even if the event took place weeks or even months prior to the investigation.

978-0-7695-4955-2/13 $26.00 © 2013 IEEE
DOI 10.1109/IMF.2013.20



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

James et al. [12] generate timestamp signatures for identifying user activities in post mortem digital investigations. They focus on generating "signatures" for the startup process of different applications and created those signatures for the following three applications: *Microsoft Internet Explorer*, *Mozilla Firefox* and *Microsoft MSN Messenger*. These signatures are based on changes to Windows Registry entries as well as changes to file system's timestamp metadata. Their experiment was divided into two phases. In the first phase, the authors repeated the startup process for each application four hundred times and monitored the accesses on files and Windows Registry entries using the *Microsoft Process Monitor*. In the second phase, they repeated the same startup actions again ten times for each application and checked which timestamp information was changed for the files identified in the first phase. For actually generating the signatures, James et al. make use of only those timestamp updates and Windows Registry modifications that were always updated.

Without naming it, James et al. [12] adapt the concept of a *fingerprint* that is known from other areas of computer security. Fingerprints arise from side channels that transport typical information that may be useful to an attacker. Probably most well-known is the technique of network fingerprinting, a method to remotely identify the producer, the version, or even configuration settings of software components, even though the system tries to hide this information. For example, the free network scanner *nmap* [13] uses a database of special characteristics of different TCP/IP implementations to unveil the version and the producer of a remote operating system. This is valuable information for an attacker, because he can then craft attacks in a more intelligent way such that they have a higher chance of working against this particular target system.

### C. Introducing Application Fingerprints

In this paper, we study the potential of *application fingerprinting* for supporting event reconstruction in digital forensics. An application fingerprint is a characteristic pattern left by a user- or system-activity (which we call an *action*) within the file system metadata. Generally, our goal is to

1) make use of such fingerprints in event reconstruction, i.e., to prove the presence or absence of known fingerprints on an analyzed system, and
2) to generate a database of such fingerprints automatically using a novel fingerprint generation system.

Like James et al. [12], we only use NTFS *MAC* timestamps for fingerprint generation, although our approach can be extended to any file system metadata that is available in the specific file system. Using only timestamp information already enables us to generate fingerprints for detecting not only the start of specific applications, but also different *actions* that can be performed with the same application. As an example, consider an email client that is used to send, edit, receive and delete email messages. If one of these actions is performed, different files are accessed by the application in different ways: some files are accessed, others are modified. This creates a pattern within the file system metadata that potentially can be fingerprinted. Unfortunately, other actions of the application might access similar files and also the concurrent execution of other tasks (e.g., of the operating system) introduces noise into the timestamp patterns. Such noise needs to be filtered during fingerprint generation. We examine whether there exist characteristic patterns — despite noise — for a set of example actions.

### D. Contributions

In this paper, we introduce the concept of forensic fingerprints based on timestamp information as available in the metadata of most file systems. We show how to automatically generate fingerprints for different actions by presenting a forensic fingerprinting framework called *Py3xF* (short for Python Forensic Fingerprinting Framework). Using Py3xF, the user is able to automatically generate fingerprints of performed actions in different applications and derive unique timestamp profiles for those actions. During a digital investigation, these fingerprints can be matched to the file system metadata of an analyzed system to identify the (non-) occurrence of these actions in the past.

We evaluated our framework by first generating fingerprints for the actions of sending and receiving emails in Mozilla Thunderbird and instant messages in ICQ. In a second step, we installed Thunderbird and ICQ on a Windows XP system and manually performed the previously fingerprinted actions in arbitrary order and noted the time when each action was performed. We then compared the resulting report of all found matches and the detected time of execution with the actually performed actions (ground truth). The evaluation shows that we are able to reconstruct all of the considered actions, however for each action only its most recent occurence can be detected.

Although we demonstrate the profiling and identification of only a limited set of applications in this work, we argue that our approach is very generic and the developed framework can be used to profile *any* activity on a computer system. In contrast to other approaches of reconstructing past events, there is no need to invest expert knowledge, e.g., about the specific file format, storage locations or internal log capability of every application. Every action that can explicitly be performed by a normal user can be fingerprinted using Py3xF (although not every action necessarily needs to leave a unique fingerprint, of course).



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

### E. Outline

This article is outlined as follows: Section II provides an overview of the important details on timestamp information stored in NTFS file systems and the update policy of accessed timestamps. In Section III, we give a more detailed introduction to the idea of application fingerprinting based on file system metadata. Section IV describes the design of a generic forensic fingerprinting framework. The technical details and specifications of our prototype implementation *Py3xF* are described in Section V. The results of the evaluation are presented in Section VI, and in Section VII, we finally conclude our work with a short summary, limitations and an outlook.

## II. Background: Timestamp information in the NTFS file system

In this work, we are focussing on file system timestamps as an example of file system metadata. Although our approach is not fixed to timestamp information, timestamps are the metadata entries with the most obvious potential to be useful in event reconstruction. In future, we could easily take other metadata, such as file size or byterun information (i.e. the actual placement of the file's content on the hard disk), into account. For a better understanding of the rest of the paper, we now have a closer look at available timestamp information in NTFS. Microsoft's *New Technology File System* (*NTFS*) is the default file system in Windows NT, 2000, XP, Vista and Windows 7 and thus the most widely used file system today. As well as most other file systems, NTFS keeps track of different timestamps that are available for each file in its metadata [6]:

- atime: Indicating the time, when the content of the file was last accessed/read.
- mtime: Indicating the time, when the content of a file was last modified/changed.
- crtime: Indicating the time, when the file was created.
- ctime: Indicating the time, when the MFT metadata entry of a file itself was last modified/updated.

Following Carrier [6], the timestamps belong to the *metadata category*, which additionally might contain other information of a file like the address of the file's content, the file size and access control information. The NTFS *Master File Table* (MFT) entries are examples of data structures in this category. Each timestamp is a 64-bit value and is measured in one-hundred nanoseconds that have passed since January 1, 1601 UTC.

It is noteworthy, that the NTFS *atime* timestamp stored in the MFT is not always current. This is because NTFS updates the on disk *atime* timestamp only when its last update was more than one hour ago. When the last update happened more recently, NTFS temporarily stores the updated *atime* timestamp in memory until it differs by more than one hour from the value currently stored on disk, before it actually writes the changes to the disk. Additionally, if all references to a file stored in memory are closed and its *atime* timestamp value has not yet been updated due to the one hour rule, the value is updated at that point in time [14].

While in Windows NT, 2000 and XP the *atime* timestamp is enabled by default and updated according to the rules stated earlier, this is not the case for Windows Vista and 7. Microsoft decided to completely disable the update of the *atime* timestamp to gain performance, since in case of a read-only operation such as viewing the file's content without modifying it, a write operation to update the *atime* timestamp can be omitted. The system administrator is able to re-enable the update of *atime* timestamps in the Windows Registry by setting the key `HKEY_LOCAL_MACHINE\SYSTEM\CurrentControlSet\Control\filesystem\NtfsDisableLastAccessUpdate` to the value `1` . The *atime* timestamp is then updated according to the previously stated rules.

After we provided the necessary background information on timestamp metadata available in NTFS file systems, the update policy for *atime* timestamps and the different presettings of the `NtfsDisableLastAccessUpdate` registry key, we introduce our approach of fingerprints based on file system metadata in the next section.

## III. Forensic Fingerprinting based on file system Metadata

In this section, we introduce our idea of generating unique fingerprints for user activities in software applications. Basically, these fingerprints consist of unique changes to the file system's metadata and are intended to support an investigator in the event reconstruction phase of a digital forensic investigation. The automated matching of such fingerprints against the metadata found in the file system of an investigated hard drive is able to provide an investigator with helpful first information on which user activities happened on the system, which applications were used, and at which time. We describe this intuitive idea more specifically in the following use case.

### A. Forensic Fingerprinting Use Case

A typical use-case scenario for digital forensic analysis is, when an investigator inspects a hard disk drive and needs to reconstruct the events that happened lately on the system. Often, those events consist of one or more consecutively performed actions.



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

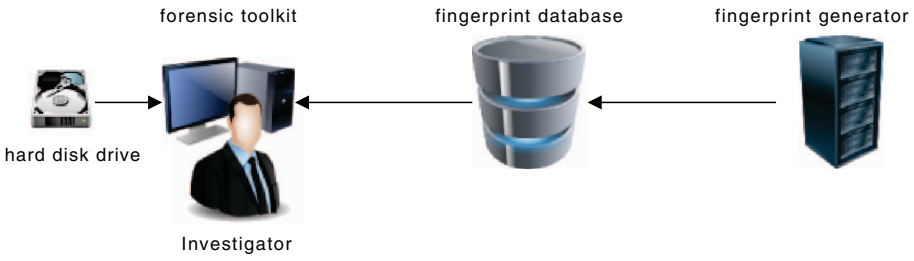


Figure 1. Use case scenario.

Much like a suspect leaving his fingerprints at the crime scene, that can then be used to link the suspect to the crime in the classical sense of forensics, software that was used to perform an action as well as the action itself leave traces that can be used to reconstruct events, too. One example of such traces are the timestamps stored in the file system for each file as described in Section II. Whenever an action is performed, the operating system updates the respective timestamps of the files that were accessed, modified or newly created during its execution. If the combination of updates to the metadata information is unique for an individual action, those unique traces can then be seen as the action's digital fingerprint. This kind of fingerprints can then, in a post-mortem analysis, be automatically matched against the metadata information contained in the file system of an evidence hard disk. Now, instead of browsing manually through thousands of lines of log files, the investigator can simply extract the file system's metadata and search for matching fingerprints from a fingerprint database as illustrated in Figure 1. If a fingerprint for a relevant action of a certain application was identified on the analyzed system, the investigator might start to dig deeper into the analysis of the affected files and the specific application.

### *B. Fingerprinting Approach*

Since the reliability of analyses performed in a post-mortem manner are considered to be higher than those of live analysis [6], we focused on using data that is actually available in post-mortem analyses. Thus, our approach aims to derive digital fingerprints based on unique changes to the file system's metadata for different actions performed in various applications. This approach has two main advantages: Firstly, the same type of data that is typically also available to the analyst during the investigation, at least if the file system is unencrypted, is also used for profiling and generating the fingerprints. The second advantage is, that this is a very generic approach, since the fingerprints merely consist of metadata information and do not depend on specific file content. This way, no specialized parsers for different file types and structures are needed.

Our approach of generating those fingerprints is, to first create application profiles for different actions that can be performed by the user of this software. Each profile basically consist of all changes to the timestamp metadata for all files that were accessed, created or modified when performing the respective action. The creation of such application profiles is done by state comparison of the file system's metadata right before and right after the execution of the action. We then match both sets of data for the extraction of all changes that happened in the meantime.

Since there are always multiple processes running on a system, even while profiling, there is a certain noise added to the application profile, such as changes to timestamp information that is not a direct result of the action itself. Therefore, it is necessary to perform multiple runs per action that is profiled, since some changes in the metadata might be the result of such other running processes or only occur under special circumstances, for example when an action is performed the first time ever on the system. We chose to generate one hundred profiles for each action, to first compare all profiles of the same action among another, in order to find those changes that happen every time an action is performed, before in a second step, we compare all profiles for different actions. The unique changes, those that only appear in the application profiles for an individual action and do not appear in any of the other profiles, are filtered out and saved as fingerprints for these actions.

As the file system stores solely the latest updated value for each timestamp, a match of a fingerprint only indicates the time an action was performed last. But since the fingerprint only consists of unique changes of timestamp information for several involved files, these relevant timestamps are only updated when performing the same action again. Therefore, when for example an email is first received, the attachment is downloaded and the email later gets deleted, the fingerprints for all these actions are still matchable. But if the user receives and deletes emails multiple times, the fingerprints would only match the last time these actions where executed.

As a proof of concept, we have designed and implemented a prototype of a forensic fingerprinting framework, that not only provides functionalities for the generation of fingerprints by automated profiling but also for matching these fingerprints



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

against file system metadata of a suspect's hard drive. Moreover, the prototype's server-client based architecture provides high scalability, enabling its usage also on the huge number of software applications found in practice. Our design decisions and implementation details are described in the following two sections.

## IV. Design of a Generic Forensic Fingerprint Generator

In the following, we give an overview of the general operation principle of our forensic fingerprinting framework and design choices such as the use of virtualization techniques to generate the application profiles, the implementation of most components as web services, as well as the use of a private cloud infrastructure.

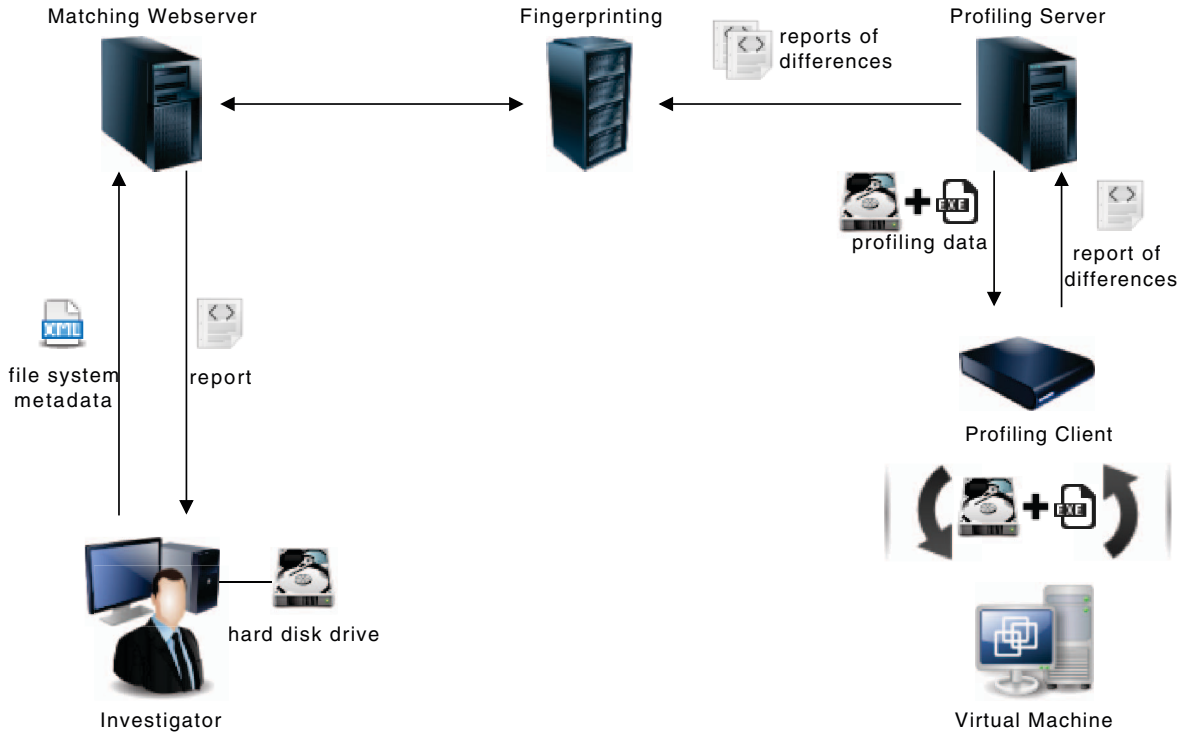


Figure 2. System Overview.

Figure 2 provides a high level overview of the system's operating principle. As shown in the picture, the main use case of the fingerprinting framework is an investigator that needs to reconstruct user activities that previously happened on a given hard disk. He therefore uploads the file system's metadata in XML format, he obtained with a digital investigation toolkit like *EnCase*[1], to the so called *Matching Webserver*. In return, the investigator receives a detailed report, containing a list of all applications and actions, that left a known digital fingerprint on the investigated hard disk. Much like in terms of classical forensics, where multiple fingerprints first need to be collected and stored in a database, before a fingerprint found at the crime scene can be used to possibly identify the criminal, we also need to gather fingerprints first. Thus, before the system is actually able to match fingerprints to any file system metadata, fingerprints need to be generated. This is done by profiling, as already introduced in Section III. Basically, the *Profiling Client* automatically and repeatedly executes an action, just the way a normal user would. On each iteration, the Profiling Client extracts all changes to the file system's timestamp metadata that are result of performing the profiled action. The Profiling Client transfers those reports to the *Profiling Server*, that collects all reports for a single action, before it transmits them to a *Fingerprinting* system. The Fingerprinting machine then extracts only those timestamp update patterns that are unique for a single action, compared to all other already profiled actions and stores them as a fingerprint.

### A. Cloud based Infrastructure

We have set up a private cloud to allow for easy scalability of the amount of profiling machines. Whenever a new profiling machine is needed, a new virtual machine is spawned in the cloud. If the cloud reaches its maximum capacity of

[1] http://www.guidancesoftware.com/encase-forensic.htm



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

virtual machines, another physical cloud compute node is added to the cloud and registered in the cloud management node automatically without manual configuration or installation of the physical machine.

### B. Scaling through Virtualization and Automation

In order to be able to generate application profiles on a large scale without the need for a physical machine per profiling run, the system operates based on virtual machines and virtual disk images. Each profiling machine is spawned within the private cloud with a preconfigured Ubuntu 12.04 Server operating system and the *VirtualBox*[2] virtualization platform preinstalled. VirtualBox provides an Application Programming Interface (API) that allows to fully automate the handling of virtual machines and their virtual disk images. This API can be used to automatically import preconfigured virtual machines that are running the application to be profiled, as well as the corresponding operating system. Those virtual machine images are paused and the corresponding machine state file containing a dump of the memory is used when importing the virtual machine. This way, a paused virtual machine can simply be resumed and triggered to execute the process of the action to be profiled from within the host machine, without having to first boot up the virtual machine and wait until it is fully functional. The process execution itself is also automated using the VirtualBox API. We therefore first copy a precompiled *AutoIt*[3] script from the host system to the virtual guest machine. AutoIt is a scripting language designed to automate user interactions with the Windows GUI. Since AutoIt scripts especially provide functionalities to automatically perform user-like interactions (such as keystrokes and mouse movements), this enables our system to automate each user activity and thus makes a very generic approach of profiling possible. In a second step, the copied AutoIt script is triggered to be run on the guest system using the API's process handling capabilities. Once the process is finished, the guest machine is automatically powered down also using the API. Afterwards, it is now possible to compare the disk image, that was modified during the process execution, with the disk image saved right before the virtual machine was imported. The usage of virtual machines and virtual disk images has the advantage that through the API and with help of AutoIt scripts it is possible to automate the process of profiling entirely and no user interaction is required. This enables the large-scale generation of application profiles and fingerprints.

One big advantage of using virtual machines is that these machines operate on virtual disk images which makes the state comparison of the file system before and after the execution of an action very easy. Instead of first having to create disk image files of physical hard drives, the virtual disk images are already stored as image files.

Another advantage of virtualization is that we are easily able to revert the machine state to a previously taken snapshot or import a backup of the virtual machine. This is necessary to ensure the profiling machine is in exactly the same initial state for each profiling run.

### C. Server-Client Architecture

The profiling and thus the generation of fingerprints, as well as the matching of fingerprints is done in a server-client based manner. In order to be able to distribute multiple profiling runs (jobs) among the profiling cloud nodes, the profiling server is mainly responsible for job management and scheduling. Both functionalities are implemented as web services. New profiling jobs may be added by authenticated users of the system. If profiling clients query the server for new jobs, the scheduler decides which job in the queue is going to be processed next according to its priority. The profiling client is where the actual profiling takes place. Whenever a client machine is idle – meaning that currently no profiling job is actually executed – it queries the profiling server for a new job. When a new job is available for the client, the server tells the client the internal URL to download the virtual machine files and the precompiled AutoIt script from.

Using web services has the main advantage that the profiling might be distributed over a vast variety of profiling clients in a cloud based infrastructure. More precisely, on computer systems that are connected to the internet, no matter where they are located, a profiling client can be installed. The functionality of matching already known fingerprints is also implemented as a web service. This way, we are not only able to provide a web interface for uploading file system metadata files but also implement according plugins for other forensic frameworks such as The Sleuthkit by providing API services.

## V. Implementation

In this section, we take a closer look only at the most relevant implementation details, since describing the whole implementation would exceed the scope of this work.

We implemented a prototype of a forensic fingerprinting framework that we called *Py3xF*. As the name already indicates, Py3xF was written in Python. We implemented Py3xF based on the open source forensic analysis program *fiwalk* by Garfinkel [15]. Fiwalk itself makes use of and was lately integrated with the well known digital forensics analyses toolkit

[2]http://www.virtualbox.org/
[3]http://www.autoitscript.com/



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

*The Sleuthkit* [6, Appendix A]. We focused on developing Py3xF to be operational on UNIX systems and to be able to fingerprint Windows applications that run on NTFS based Microsoft Windows operating systems; especially on Windows XP and Windows 7, which are well known and were at the time of writing Microsoft's most widely used operating systems. Since fiwalk and The Sleuthkit are able to operate and extract the necessary metadata of different UNIX file systems (like EXT 2/3/4 or UFS 1/2) too, we implemented our framework to also be able to work on data based on those file systems.

Since we have two different main use-cases for Py3xF that are described in the following, we decided to implement two different components that also might be run on totally different machines. The first use-case is the automated generation of application profiles and fingerprints. This functionality was implemented in the *Profiler* component of Py3xF. The second use-case is the matching of hard disk images against a database of previously generated fingerprints and the reporting of all found matches. We implemented that functionality in Py3xF's *Matcher* component.

### A. The Profiler Component

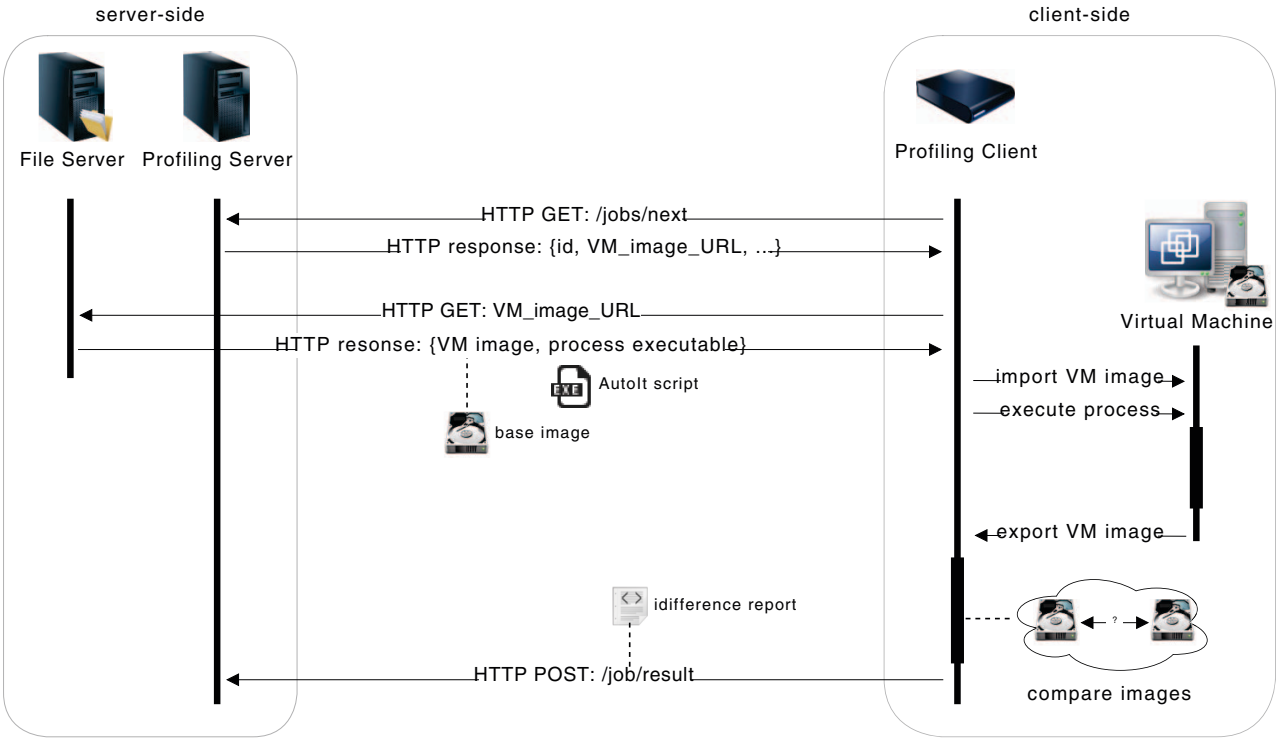


Figure 3. Profiling Server-Client Communication.

Py3xF's Profiler component is split up to a profiling server and one or more profiling clients. The profiling server is responsible for job management, job scheduling, as well as the generation of fingerprints. The profiling clients, on the other side, are the worker nodes; they do the actual work of profiling. Figure 3 illustrates the server-client communication during a single profiling run. We decided to implement Py3xF based on HTTP since HTTP provides us with all required request methods like *GET*, *POST*, *PUT* and *DELETE* for a server-client based communication. The communication for a profiling run is started by the profiling client, whenever currently no job is being executed (idle-state). The client, in this case, queries the server for a new job; if currently no job is available, the response of the server contains an integer number that tells the client to wait for this amount of seconds before the next job query. On the other hand, if a job is available, the server's job scheduler will pick the next job according to its priority and return the job details to the client. The job details mainly are a unique job id, a URL of the virtual machine files and the name of an AutoIt executable. The different virtual machine files and AutoIt scripts are hosted on an internal fileserver that is only reachable from machines in the same subnet. Upon receiving a new job, the client parses the job details and fetches the virtual machine image and the according AutoIt executable. Using the VirtualBox API, we have implemented functionalities to automatically import the virtual machine files, power on the virtual system, copy the AutoIt executable to the guest system and execute the the AutoIt script. When the process execution has finished, the virtual machine is powered off and the disk image containing the new, modified state of



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

the file system, is then exported on the client. Now, with the help of fiwalk's `idifference.py`[4] Python module, we then automatically extract the changes of the metadata information by comparing the images containing the file system in the state before and after the action was executed. The resulting idifference report, among other information, contains a list of all changes made to the timestamp metadata in the meantime. This report is then transferred back to the profiling server.

Since each job consists of multiple profiling runs, the server collects all reports before further processing an individual report. When a job is completely finished (meaning all profiling runs have successfully finished and returned the according idifference report), the server then parses the reports and stores their content to a MySQL database.

The query, that is used to extract the unique modifications of timestamp information in the file system's metadata, is shown in Listing 1. Here, we select those profile components that are unique to the given action (lines 1–3) and that appear in each of their idifference reports (lines 12–16). We therefore require, that all changes of the same timestamp information for the same file, found in applications profiles for different actions, are filtered (lines 4–10) and only the unique changes are returned.

As stated in Section III, in some cases it is helpful to exclude specific actions from the filter (lines 4–10), especially if two different actions do not have a unique fingerprint but the combination of both has. Therefore, we implemented a functionality, that enables the user to specify a list of actions that should not be filtered. We then simply extend the query to ignore the named actions (line 9).

Listing 1. SQL query to extract unique modifications of timestamps.

```
1  SELECT max(run), count(*), path, changed, action
2  FROM profiles
3  WHERE profiles.action='SOME_ACTION'
4          AND profiles.path NOT IN (
5                  SELECT path
6                  FROM profiles AS profiles_2
7                  WHERE profiles_2.action != 'SOME_ACTION'
8                          AND profiles.changed = profiles_2.changed
9                          $SUBQUERY_EXCLUDES
10         )
11 GROUP BY path, changed
12 HAVING count(*) > (
13         SELECT MAX(run)
14         FROM profiles AS profiles_3
15         WHERE profiles_3.action = 'SOME_ACTION'
16 )
17 ORDER BY id ASC;
```

After several application profiles are stored, we are then able to query the database for only those changes that happened in each profiling-run of a single action and did not appear in a profiling-run of any other action. The data returned by the query is the fingerprint for the individual action and is written back to the database. Table I shows an example fingerprint for the action of receiving a message in *ICQ 7M*[5] that is running on a Microsoft Windows XP machine. The *Path and File* column tells which file's timestamp information was updated while performing the action. The *Updated Timestamp* column indicates which of the NTFS timestamps (*atime*, *mtime*, *crtime*, *ctime*) was updated.

### B. The Matcher Component

Py3xF's Matcher component is responsible for matching a given disk image to the database of fingerprints and for generating a report for all found matches. We implemented the Matcher component to first extract the file system's metadata with the help of fiwalk. Fiwalk generates an XML file containing the metadata including the necessary timestamp information. Additionally, we have implemented a parser module for fiwalk XML files as well as EnCase XML reports that extracts the timestamp information for each file that is stored on the hard disk image which is being analyzed. Based on the list of files and their timestamp information, we then query the fingerprint database comparing if the pattern of changes of the timestamps stored for each fingerprint match for the given file and its timestamps. In more detail, we check for each file that is listed in the file system's metadata if its filename and path belongs to a known fingerprint stored in the database. Since fingerprints might consist of multiple path and filenames, we add new matches to a list of already found matches. Matches that belong to a fingerprint that was already matched by a path and filename of another file stored on the disk, are simply appended to the list entry for the previously found match. For each match, we not only store the filenames and their paths but also their timestamp value of the timestamps that belong to the fingerprint. After each file contained in the XML report

[4]http://afflib.org/software/fiwalk
[5]http://www.icq.com/



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

| Path and File | Timestamps |
|---|---|
| Program Files/ICQ7M/packages/ICQ/ConfigFiles/SpamFilter.xml | *atime* |
| Program Files/ICQ7M/sounds/Default/Scheme.xml | *atime* |
| Program Files/ICQ7M/imApp/content/MUICore/ToasterDlg.gadgets.box | *atime* |
| Program Files/ICQ7M/imApp/content/MUICore/ToasterLightDlg.box | *atime* |
| Program Files/ICQ7M/imApp/resources/en-US/ToasterDlg.dtd | *atime* |
| Program Files/ICQ7M/imApp/theme/IMAGES/Common/ icqToasterWindow/inputToaster.png | *atime* |
| Program Files/ICQ7M/imApp/theme/MUICore/ToasterDlg.style.box | *atime* |
| Program Files/ICQ7M/imApp/theme/MUIUtils/Dialog.style.box | *atime* |
| Program Files/ICQ7M/packages/german/language/de-DE/ToasterDlg.dtd | *atime* |
| Program Files/ICQ7M/sounds/Default/IncomingIM.mp3 | *atime* |
| WINDOWS/system32/l3codecx.ax | *atime* |
| WINDOWS/system32/quartz.dll | *atime* |

Table I
EXAMPLE FINGERPRINT FOR RECEIVING AN ICQ MESSAGE.

has been compared to the known fingerprints a report is generated. The report then states which fingerprints were matched and the time of execution of the matched action. The time of execution is set to the minimal timestamp value among all timestamps of files that belong to the fingerprint. For each match listed in the report, we additionally calculate a percentage value indicating if all parts of the fingerprint were matched.

Since a fingerprint can consist of multiple updated timestamp values for different files, some updates might happen somewhat later then others. If, for example an action takes longer than a second to be executed, the individual timestamps will most likely differ from one another. Thus, when matching an action to a certain time, not all timestamps contained in the fingerprint are likely to have the exact same value. Therefore, we implemented a functionality, that enables the user to specify a threshold value in seconds that timestamps for the action are allowed to differ from the reported time of execution and still be matched as one hundred percent. Since, all found matches are reported back, even those whose individual timestamp values (partly) exceed the provided threshold, the investigator might best guess the threshold value.

We implemented three different types of report functionalities. Py3xF generates a plain text report for command line usage, XML reports (see Listing 2) for further processing the results and HTML reports (see Figures 4 and 5) that are actually transformed XML reports using XSLT stylesheets. Each report contains the same information as the others. An extract of an example XML report is shown in Lisiting 2. Each found match contains the following information:

- The action tag, which is a unique and descriptive identifier for the performed action (line 2, `<action>` ).
- The time of execution, indicating when an action was last performed on the investigated system (line 3, `<date>` ).
- The calculated accuracy for the match as a percentage value and the information how many files and timestamps of the fingerprint were actually found and matched (lines 4-8, `<accuracy>` ).
- A list of correctly matched parts of the fingerprint. Each child node has the attributes *path* and *timestamp*, that indicate which timestamp of which file was matched correctly (lines 9-12, `<correctTimestamps>` ).
- A list of timestamps that were not matched correctly. Each child node contains, in addition to the child nodes of correctly matches timestamps, an *offset* and a *time* attribute, indicating the number of seconds the timestamp value varies from the time of execution and the value of the timestamp respectively (lines 13-16, `<wrongTimestamps>` ).
- A list of additional files that should have been stored on the system according to the fingerprint, but could not be found. Each missing file is added to the list in the form of <missingFile>/path/to/ file . ext</missingFile> (line 17, `<missingFiles>` ).

To ease the process of matching for the user as much as possible, we implemented a simple web interface that can be used to upload fiwalk and EnCase XML representations of the file system metadata. The uploaded file is processed automatically and the HTML report is displayed to the user. Additionally, the users are able to download an XML based report (as shown in Listing 2) and to store their reports on the Matching Webserver.

Listing 2. XML report of results of fingerprint matching.

```
1  <match>
2    <action>THUNDERBIRD_RECEIVE_EMAIL</action>
3    <date>Thu, 25 Oct 2012 11:46:15</date>
4    <accuracy>
5      <percentage>40.00</percentage>
6      <files>3/3</files>
7      <timestamps>2/5</timestamps>
```



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

```
  </accuracy>
  <correctTimestamps>
    <correctTimestamp path="WINDOWS/Media/Windows␣XP␣Notify.wav" timestamp="atime"/>
    ...
  </correctTimestamps>
  <wrongTimestamps>
    <wrongTimestamp offset="71" path="Documents␣and␣Settings/user/Application␣Data/Thunderbird/
        Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/␣popstate.dat" time="1351158446" timestamp=
        "mtime"/>
    ...
  </wrongTimestamps>
  <missingFiles/>
</match>
```

## VI. Evaluation

For the evaluation of our proof of concept forensic fingerprinting framework, we used Py3xF in a case study as follows. At first, we generated action profiles for typical actions that can be performed with an email client and an instant messenger. We then derived unique fingerprints for each action and finally matched a HDD that was specially prepared for this purpose against those fingerprints.

For the process of profiling and generating fingerprints, we set up a Microsoft Windows XP Professional 32-Bit virtual machine and installed an email client and an instant messenger. We saved a backup of this base image of the virtual machine to later be able to compare the modified virtual disk images with this image. We chose Mozilla Thunderbird as an example for an email client and ICQ 7M as instant messenger, since both applications are freely available and commonly used. As typical actions that are usually performed within these applications, we determined sending and receiving emails and instant messages respectively.

First, we used Py3xF's Profiler component to automatically generate one hundred runs for each action, resulting in altogether four hundred virtual disk image files. With help of fiwalk's `idifference.py`, Py3xF generated the idifference log files that contain all timestamp values that have been updated on each of those disk images compared to the base image. Py3xF stored the changes of the file system's timestamp metadata in a database table, which then is used to generate the fingerprints.

| Path | Timestamps |
|---|---|
| `Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/msgFilterRules.dat` | *atime* |
| `Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat` | *atime, mtime, ctime* |
| `WINDOWS/Media/Windows XP Notify.wav` | *atime* |

Table II
Fingerprint for receiving an email in Mozilla Thunderbird (`THUNDERBIRD_RECEIVE_EMAIL`).

With the query described in Section V-A and shown in Listing 1, we extracted all unique changes to the timestamp information for each action and stored them as fingerprints in another MySQL database table. Example fingerprints are shown in the Tables I and II.

For the purpose of matching the generated fingerprints, we used a backup of the virtual machine that was set up and used for profiling. As a test case, we performed the following actions on this virtual machine on October the 25, 2012 and denoted their times of execution:

- 11:39 - Receiving an incoming message in ICQ 7M (`ICQ_RECEIVE_MESSAGE`)
- 11:39 - Sending an outgoing ICQ message (`ICQ_SEND_MESSAGE`)
- 11:43 - Sending an email in Mozilla Thunderbird (`THUNDERBIRD_SEND_EMAIL`)
- 11:46 - Receiving a new email in Mozilla Thunderbird (`THUNDERBIRD_RECEIVE_EMAIL`)

Afterwards, we powered down the virtual machine and used fiwalk to export the file system's metadata to an XML file. We used this XML report as an input for Py3xF Matcher component by uploading it to the Matching Webserver. We configured Py3xF with a threshold value (discussed in Section V-B) of ten seconds, since we assumed each action to be fully executed within this time frame.

Figure 4 shows an overview of all matches found by Py3xF for the given file system metadata. As we can see, all fingerprinted actions were actually matched with the correct time of execution. In fact, except for `THUNDERBIRD_RECEIVE_EMAIL`,



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

## Matches Overview

show/hide details

| Accuracy | Action | Date | Type |
|---|---|---|---|
| 100.00% | ICQ_RECEIVE_MESSAGE | Thu, 25 Oct 2012 11:39:34 | Action Match (0) |
| 100.00% | ICQ_SEND_MESSAGE | Thu, 25 Oct 2012 11:39:36 | Action Match (0) |
| 40.00% | THUNDERBIRD_RECEIVE_EMAIL | Thu, 25 Oct 2012 11:46:15 | Action Match (0) |
| 100.00% | THUNDERBIRD_SEND_EMAIL | Thu, 25 Oct 2012 11:43:31 | Action Match (0) |

Figure 4. Overview of found fingerprint matches in the first test case, as produced by Py3xF.

| | | | |
|---|---|---|---|
| 40.00% | THUNDERBIRD_RECEIVE_EMAIL | Thu, 25 Oct 2012 11:46:15 | Action Match (0) |

**Correct timestamps for:**

- Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/msgFilterRules.dat
- WINDOWS/Media/Windows XP Notify.wav

**Wrong timestamps for:**

- atime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 71 sec)
- ctime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 71 sec)
- mtime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 71 sec)

Figure 5. Detailed information on the match found for `THUNDERBIRD_RECEIVE_EMAIL` in the first test case.

all matches show an accuracy value of one hundred percent, indicating that all parts of the respective fingerprints were found and all timestamp values are within the predefined ten seconds range. In general, a matching fingerprint indicates that the corresponding application is or at least previously was installed on the investigated system and the individual action was performed last at the reported time. If a fingerprint stored in the database cannot (partly) be matched to the investigated file system, this indicates that there was no evidence found in the file system metadata that the respective action was ever performed on the system since it was last reinstalled.

`THUNDERBIRD_RECEIVE_EMAIL` shows only an accuracy value of 40% meaning that only two out of altogether five timestamp values were matched at the given time. Figure 5 depicts detailed information for this match. As we can see, the *atime*, *ctime* and *mtime* timestamp of the `popstate.dat` file were not updated within the ten seconds range, but were updated 71 seconds after the time of execution. This indicates, that completing the whole `THUNDERBIRD_RECEIVE_EMAIL` action process, took actually around 71 seconds. Thunderbird uses the `popstate.dat` file to enable functionalities like fetching email header files only or leave mail copies on the server after downloading them for POP3 accounts [16]. Since this file is only needed when using the POP3 protocol for fetching mails from a server, this indicates that the fingerprint is only valid for POP3 Thunderbird accounts. Other email accounts, that use for example the IMAP protocol for server-client



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

communication, need to be profiled and fingerprinted separately. The match would have been a one hundred percent match if we had configured Py3xF with a threshold value greater or equal than 71 seconds.

What can also be seen from Table II and the details of the match as given in Figure 5, is that some file paths, e.g. the file `popstate.dat`, contain the name of the user's Thunderbird profile (`/Profiles/t4oa3jeg.default/`). This profile folder is automatically generated when using Thunderbird for the first time or by adding another email profile in Thunderbird. Since this profile name varies for different users and different systems, we would not have been able to match these parts of the fingerprints, if we did not use the same system as for profiling. We discuss this problem in more detail in Section VII-A.

In a second test case, we performed the same actions that we already regarded in the first test case, but this time we performed each action twice. Again, on October 25, 2012 we ran the following actions and denoted their time of execution:

- 13:11 - `THUNDERBIRD_SEND_EMAIL`
- 13:12 - `ICQ_SEND_MESSAGE`
- 13:12 - `THUNDERBIRD_RECEIVE_EMAIL`
- 13:13 - `ICQ_RECEIVE_MESSAGE`
- 13:13 - `THUNDERBIRD_SEND_EMAIL`
- 13:15 - `ICQ_SEND_MESSAGE`
- 13:16 - `ICQ_RECEIVE_MESSAGE`
- 13:16 - `THUNDERBIRD_RECEIVE_EMAIL`

Again, we used Py3xF's Matcher to generate a report that details all found matches, their time of execution and the calculated accuracy value. Figure 6 depicts the Py3xF's report generated for the second test case. There is one thing conspicuous about the report, which is that it only contains four matches despite we executed each of the four actions twice. This is because NTFS only stores the latest updated timestamp values for each file, as already stated in Section II. Therefore, Py3xF is only able to match each action once, even if these action were executed multiple times.

Matches Overview

show/hide details

| Accuracy | Action | Date | Type |
|---|---|---|---|
| 100.00% | ICQ_RECEIVE_MESSAGE | Thu, 25 Oct 2012 13:13:52 | Action Match (0) |
| 100.00% | ICQ_SEND_MESSAGE | Thu, 25 Oct 2012 13:11:56 | Action Match (0) |
| 40.00% | THUNDERBIRD_RECEIVE_EMAIL | Thu, 25 Oct 2012 13:12:37 | Action Match (0) |
| 100.00% | THUNDERBIRD_SEND_EMAIL | Thu, 25 Oct 2012 13:14:40 | Action Match (0) |

Figure 6. Overview of found fingerprint matches in the second test case.

Thus, we would expect, that the report states the last four actions performed between 13:13 and 13:16. Looking at the reported execution time shown in Figure 6, we can tell, that only the match for `THUNDERBIRD_SEND_EMAIL` actually goes with that expectation. The other matches are reported on their first time of execution. For both ICQ actions this is the case, because those fingerprints only contain changes of the *atime* timestamp. In Section II, we have seen that NTFS updates the *atime* timestamps only on an hourly basis. Since the second execution for both actions happened just minutes after the first, the respective *atime* timestamp did not get updated. The fingerprints for both actions of Thunderbird also contain *mtime* and *ctime* timestamps. In NTFS, these timestamps get updated right away, when the file or MFT entry was actually modified. The according *atime* timestamp for these files is also updated in this case, even if it was already updated under an hour ago. Therefore, the timestamp for `THUNDERBIRD_SEND_EMAIL` is matched correctly, since it consists of only the *atime*, *mtime* and *ctime* timestamp for a single file. So, when this file is modified, all three timestamps are updated immediately. Looking at the details for `THUNDERBIRD_RECEIVE_EMAIL` shown in Figure 7, we can tell, that unlike in the first test case, the timestamps for `popstate.dat` were not updated around 71 seconds but 272 seconds later. The additional 201 seconds are exactly the time that has passed between the first execution at 13:12 and the second execution at 13:16. But since the



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

fingerprint for this action additionally consists of changes to the accessed timestamp for two other files, which did not get updated due to the one hour rule for NTFS *atime* timestamps, this match is reported to be last executed at 13:12 instead of 13:16. With some background knowledge about NTFS and the timestamp for this action, the investigator might even conclude that in this case, the action was run at least twice within that timeframe.

| | | | |
|---|---|---|---|
| 40.00% | THUNDERBIRD_RECEIVE_EMAIL | Thu, 25 Oct 2012 13:12:37 | Action Match (0) |

**Correct timestamps for:**

- Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/msgFilterRules.dat
- WINDOWS/Media/Windows XP Notify.wav

**Wrong timestamps for:**

- atime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 272 sec)
- ctime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 272 sec)
- mtime for: Documents and Settings/user/Application Data/Thunderbird/Profiles/t4oa3jeg.default/Mail/pop.googlemail.com/popstate.dat (+ 272 sec)

Figure 7. Detailed information on the match found for `THUNDERBIRD_RECEIVE_EMAIL` in the second test case.

## VII. Conclusion

In times of increasing numbers of digital forensic investigations, as well as a rapidly increasing amount of digital data, we consider the pure manually performed event reconstruction process as inefficient and expensive. Therefore, we presented a prototype of a forensic fingerprinting framework called Py3xF. We have shown that with Py3xF it is possible to generate profiles of user activities in software applications. Furthermore, based on those application profiles, Py3xF is able to automatically derive unique fingerprints – if such exist – based on timestamp information stored in the file system metadata. Moreover, Py3xF is able to extract the file system metadata of given disk image files or parse XML representations of already extracted metadata information generated by popular forensic toolkits like EnCase or The Sleuthkit/fiwalk. This file system metadata can then be matched against a set of previously generated fingerprints. Py3xF also creates a detailed report of all found matches of fingerprints in human readable text format, XML format and HTML reports using XSLT.

### A. Limitations

Since Py3xF operates mainly on timestamp metadata, it is prone to manipulation of these timestamp values. If timestamps are intendedly changed, deleted or set back by an attacker, Py3xF is unable to detect such manipulations and will report a matched fingerprint with this falsified time information.

Today, Py3xF is unable to automatically identify temporary file or path names or ones that are system or OS dependent, like paths that contain the users names (e.g. `C:\Users\[username]\...`). Thus, we implemented a functionality in Py3xF, that matches paths and files based on a regular expressions. This way, temporary or system/OS dependent components in path can be expressed by an regular expression for each fingerprint. But since Py3xF fails to automatically identify those paths and file names, it is not able to automatically generate such regular expression for those specific paths. In particular, this means that the operator of the profiling system has to identify such paths and generate the regular expressions by himself in order to be able to fully match such fingerprints against other systems than the one he used to make the profiles with.

Up to this point, we have only implemented a proof of concept system and evaluated a small set of applications and actions. Fingerprinting and matching a broad set of actions in various applications running in different operating systems is yet to be evaluated.

Of course, not every action necessarily leaves distinguishable fingerprints in timestamp metadata. If there are two or even more actions that change files in the same way, our tool obviously would not be able to distinguish. Rather, we would



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

find that none of them is fingerprintable in the first place. In such a case, we could make use of the exclude-list feature of the framework (see Section V-A) to calculate a common fingerprint of both actions together, which then still could be differentiated against other actions. Despite this limitation, in practice surprisingly many actions produce unique fingerprints: For example, we applied our concept to a real-world case, where a suspect had installed a wiping software in a company environment and it was asked, whether the person used it to wipe files or not. Although the manufacturers of the tool advertised the plausible deniability of the tool's usage, we were able reconstruct both, the time of the last run of the program and the time of the last wiping activity using our prototype.

### *B. Future Work*

Currently, fingerprints generated by Py3xF consist of file names, their full path, as well as the timestamp information that was altered while performing the according action. This is difficult, since on different systems path components as the username or temporary file names are likely to be different from those stored in the fingerprints. Thus, it is our intention to further improve the process of fingerprint generation by applying machine learning techniques. Machine learning might not only solve the issue of different path and file names in different systems but also be more scaleable than the database driven approach we presented earlier, when it comes to the generation of large sets of fingerprints.

Additionally, we are going to implement features to exploit other kinds of sources for fingerprint generation and fingerprint matching, such as journal files of journaling file systems. We expect, that with the additional information stored in a journaling file system, it will be possible to not only match the latest occurrence of a fingerprint but also previous ones.

Another future task is the extension of Py3xF to support profiling, fingerprinting and matching mechanisms for smartphone applications running on iOS or Android operating systems, since such devices a very common nowadays and might also contain valuable data for forensic investigations.

## ACKNOWLEDGMENTS

This work was supported by Deutsche Forschungsgemeinschaft (DFG) within the project "Dynamo – Dynamische Erkennung von Schadcode mit Techniken des maschinellen Lernens".

## REFERENCES

[1] S. Garfinkel, "Digital forensics research: The next 10 years," *Digital Investigation*, vol. 7, pp. 64–73, 2010.

[2] R. Golden G. and V. Roussev, "Next-generation digital forensics." *Commun. ACM*, vol. 49, no. 2, pp. 76–80, 2006. [Online]. Available: http://dblp.uni-trier.de/db/journals/cacm/cacm49.html#RichardR06

[3] P. Turner, "Selective and intelligent imaging using digital evidence bags," *Digital Investigation*, vol. 3, pp. 59–64, 2006.

[4] J. Stüttgen, A. Dewald, and F. C. Freiling, "Selective imaging revisited," in *7th International Conference on IT Security Incident Management & IT Forensics (IMF)*. IEEE, 2013.

[5] E. Casey, *Digital Evidence and Computer Crime: Forensic Science, Computers, and the Internet*, 3rd ed. Academic Press, 2011.

[6] B. Carrier, *File System Forensic Analysis*. Addison-Wesley Professional, 2005.

[7] B. Carrier and E. H. Spafford, "Defining event reconstruction of digital crime scenes," *Journal of forensic sciences*, vol. 49, no. 6, pp. 1291–8, 11 2004.

[8] Y. Zhu, P. Gladyshev, and J. James, "Using shellbag information to reconstruct user activities," *Digital Investigation*, vol. 6, Supplement, no. 0, pp. S69 – S77, 2009, the Proceedings of the Ninth Annual DFRWS Conference. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1742287609000413

[9] H. Carvey, "The windows registry as a forensic resource," *Digital Investigation*, vol. 2, no. 3, pp. 201 – 205, 2005. [Online]. Available: http://www.sciencedirect.com/science/article/pii/S1742287605000587

[10] K.-P. Chow, F. Y. W. Law, M. Y. K. Kwan, and P. K. Y. Lai, "The rules of time on ntfs file system." in *SADFE*, M.-Y. Huang and D. A. Frincke, Eds. IEEE Computer Society, 2007, pp. 71–85. [Online]. Available: http://dblp.uni-trier.de/db/conf/sadfe/sadfe2007.html#ChowLKL07

[11] J. Grier, "Detecting data theft using stochastic forensics," *Digit. Investig.*, vol. 8, pp. S71–S77, Aug. 2011. [Online]. Available: http://dx.doi.org/10.1016/j.diin.2011.05.009

[12] J. I. James, P. Gladyshev, and Y. Zhu, "Signature based detection of user events for post-mortem forensic analysis," in *Digital Forensics and Cyber Crime*, ser. Lecture Notes of the Institute for Computer Sciences, Social Informatics and Telecommunications Engineering. Springer Berlin Heidelberg, 2011, vol. 53, pp. 96–109.



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.

[13] G. *Fyodor* Lyon, "Nmap network scanning - the official nmap project guide to network discovery and security scanning," 2009, http://nmap.org/book/osdetect.html.

[14] Microsoft Windows Team, *Microsoft Windows XP Professional resource kit*, ser. Pro-Resource Kit. Microsoft Press, 2003, no. 1.

[15] S. Garfinkel, "Automating disk forensic processing with sleuthkit, xml and python," in *Proceedings of the 2009 Fourth International IEEE Workshop on Systematic Approaches to Digital Forensic Engineering*, 2009, pp. 73–84.

[16] "Popstate.dat - MozillaZine Knowledge Base," http://kb.mozillazine.org/Popstate.dat, 2011.



Authorized licensed use limited to: Jain University. Downloaded on March 26,2026 at 09:22:04 UTC from IEEE Xplore. Restrictions apply.